

Лабораторная работа №7

Автор: Терин Ярослав

Группа: 6203-010302D

Задание 1

В этом задании требовалось адаптировать программу под цикл `for-each` для табулированных функций. Для этого мы изменили интерфейс `TabulatedFunction`, добавив к нему наследование от `Iterable<FunctionPoint>`. Это позволило использовать объекты табулированных функций в циклах `for-each` для перебора точек. Затем мы реализовали метод `iterator` в классах `ArrayTabulatedFunction` и `LinkedListTabulatedFunction`, который возвращает анонимный класс итератора. В классе `ArrayTabulatedFunction` итератор работает с внутренним массивом точек `points_arr`. Он отслеживает текущий индекс `currentIndex` и при вызове `next` возвращает копию текущей точки, после чего увеличивает индекс. В классе `LinkedListTabulatedFunction` итератор работает со связанным списком, который хранит текущий узел `currentNode` и счетчик посещенных узлов `nodesVisited`, при каждом вызове `next` переходя к следующему узлу. Для защиты инкапсуляции итераторы возвращают не сами внутренние объекты точек, а их копии. Ниже представлено тестирование в “мейн” по образцу из задания.

Точки функции:

```
(0.0; 0.0)
(1.0; 1.0)
(2.0; 4.0)
(3.0; 9.0)
(4.0; 16.0)
```

Точки через явный итератор:

```
(0.0; 0.0)
(1.0; 1.0)
(2.0; 4.0)
(3.0; 9.0)
(4.0; 16.0)
```

Задание 2

В этом задании мы сделали “фабрику функций”. Для этого создали интерфейс `TabulatedFunctionFactory` с тремя методами `createTabulatedFunction`, соответствующими трем конструкторам классов табулированных функций. Затем добавили в классы `ArrayTabulatedFunction` и `LinkedListTabulatedFunction` вложенные публичные классы-фабрики `ArrayTabulatedFunctionFactory` и `LinkedListTabulatedFunctionFactory`, которые реализуют этот интерфейс и создают объекты соответствующих типов. В классе `TabulatedFunctions` мы добавили статическое поле `factory` типа `TabulatedFunctionFactory`, инициализированное фабрикой `ArrayTabulatedFunction` по умолчанию, и метод `setTabulatedFunctionFactory` для его изменения во время выполнения программы. Ниже представлено тестирование в “мейн” по образцу из задания.

```
class functions.ArrayTabulatedFunction
class functions.LinkedListTabulatedFunction
class functions.ArrayTabulatedFunction
```

Задание 3

В это задании были добавлены новые рефлексивные методы. Для этого мы перегрузили три метода `createTabulatedFunction`, добавив к каждому параметр типа `Class`, который указывает, объект какого класса нужно создать. Эти методы принимают только классы, реализующие интерфейс `TabulatedFunction`. В них мы сначала проверяем, что переданный класс действительно реализует нужный интерфейс с помощью `TabulatedFunction.class.isAssignableFrom`. Затем с помощью рефлексии ищем конструктор с соответствующими параметрами. Найденный конструктор используется для создания объекта через `newInstance`. Также мы перегрузили методы `tabulate`, `readTabulatedFunction` и `inputTabulatedFunction`, чтобы они тоже могли принимать параметр `Class` и создавать объекты указанного типа через рефлексия. Для устранения дублирования инициализации массива также создали отдельный метод `createValuesArray`. Ниже представлено тестирование в “мейн” по образцу из задания.

```
class functions.ArrayTabulatedFunction
{(0.0; 0.0), (5.0; 0.0), (10.0; 0.0)}
class functions.ArrayTabulatedFunction
{(0.0; 0.0), (5.0; 5.0), (10.0; 10.0)}
class functions.LinkedListTabulatedFunction
{(0.0; 0.0), (5.0; 25.0), (10.0; 100.0)}
class functions.LinkedListTabulatedFunction
{(0.0; 0.0), (0.3141592653589793; 0.3090169943749474), (0.6283185307179586; 0.5877852522924731), (0.9424777960769379; 0.8090169943749475), (1.2566370614359172;
```